

[22] On the Importance of Initialization and Momentum in Deep Learning

저자: Ilya Sutskever, James Martens, George Dahl, Geoffrey Hinton (University of Toronto)

학회/년도: ICML 2013 (PMLR Vol. 28)

분량: 약 14페이지

역할군: (D) 이론적 기반

요약

이 논문은 신경망의 학습에서 **초기화(initialization)**와 **모멘텀(momentum)**의 역할을 다룬 기념비적 논문이다. 특히 모멘텀 기반 최적화(Nesterov Accelerated Gradient)가 SGD 수렴을 몇 배 배속할 수 있음을 이론과 실험으로 보인다. Exqutor의 적응적 샘플링 알고리즘(수식 3~5)이 카디널리티 추정 오차를 보정할 때 **모멘텀 기반 조정**을 사용하는데, 이 논문의 이론적 배경을 제공한다. 딥러닝 최적화의 고전 논문으로서, Sutskever와 Hinton이라는 거장의 연구로 학술적 신뢰도가 높다. Exqutor가 왜 단순 피드백이 아니라 모멘텀 기반 조정을 사용했는지 이해하는 핵심 논문이다.

상세분석

22.1 해결하고자 하는 문제: SGD의 수렴 지연

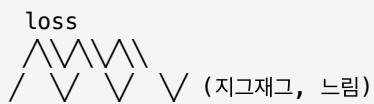
문제: 지그재그 수렴 (Zigzag Convergence)

신경망을 학습할 때 **확률적 경사 하강법(SGD)**이 느리게 수렴하는 현상이 있다:

이상적 수렴:

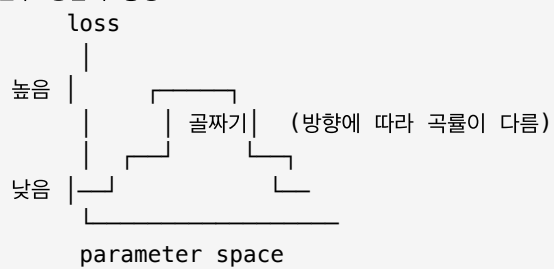


실제 SGD 수렴:



원인: 손실 함수의 지형이 방향마다 다르다

매개변수 공간의 풍경:



"길고 좁은 골짜기" 형태:

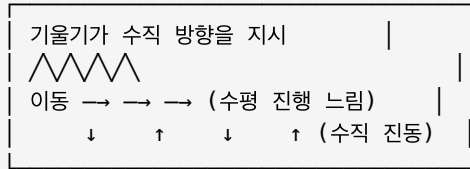
- 수평 방향 (골짜기 바닥): 경사 완만
- 수직 방향 (골짜기 벽): 경사 급함

SGD의 문제:

SGD는 현재 위치의 기울기(∇L)를 따라 이동:

$$\theta(t+1) = \theta(t) - \alpha \cdot \nabla L(\theta(t))$$

골짜기 바닥을 따라가야 하지만,
기울기는 수직 방향(벽) 쪽을 더 크게 지시:



결과: 골짜기 바닥으로 내려가지 못하고 벽을 왔다갔다함

구체적 수치 예시

간단한 이차 함수:

$$L(x, y) = 100x^2 + y^2$$

기울기:

$$\nabla L = [200x, 2y]$$

시작점: (1, 1)

학습률: $\alpha = 0.01$

SGD 진행:

Iteration 0: (1, 1)

$$\nabla L = [200, 2]$$

$$\text{이동} = [-2, -0.02]$$

Iteration 1: (-1, 0.98)

$$\nabla L = [-200, 1.96]$$

$$\text{이동} = [2, -0.0196]$$

Iteration 2: (1, 0.96)

계속 x 좌표가 -1, 1, -1, ... 반복

y는 천천히 0으로 수렴

결론:

- x 축: 지그재그 (비효율)
- y 축: 느린 수렴
- 전체: 매우 느린 수렴 (수백 이터레이션 필요)

22.2 핵심 기여: 모멘텀의 원리

모멘텀 기본 공식

모멘텀 없는 SGD:

$$\theta(t+1) = \theta(t) - \alpha \cdot \nabla L(\theta(t))$$

모멘텀이 있는 SGD:

$$v(t) = m \cdot v(t-1) + \nabla L(\theta(t)) \quad \text{-- "속도" 벡터 누적}$$

$$\theta(t+1) = \theta(t) - \alpha \cdot v(t) \quad \text{-- 누적 속도로 이동}$$

또는:

$$v(t) = m \cdot v(t-1) + (1-m) \cdot \nabla L(\theta(t)) \quad \text{-- 정규화 버전}$$

변수 해석:

- $v(t)$: 현재 "속도" (이전 이동의 관성)
- m : 모멘텀 계수 (보통 0.9) — 이전 속도를 얼마나 유지할지
- $\nabla L(\theta(t))$: 현재 기울기
- α : 학습률

직관적 이해: 공을 굴리기

모멘텀 없음:

매 순간 기울기 방향으로만 이동

↓ (기울기에 좌우됨)

↓

↓ (벽을 따라 튀어나감)

↗ ↘ ↗ ↘ (지그재그)

모멘텀 있음:

이전 이동의 관성을 유지

↓ (기울기 + 관성)

↓ (수직 성분은 상쇄)

↓ (수평 성분은 누적)

→ → → → (일정 방향으로 수렴)

공학적 유추:

기울기 = 순간 힘

모멘텀 = 질량 × 속도 (뉴턴의 제2법칙)

공은 벽에 부딪혀도 이전 속도 때문에 계속 진행

지그재그 운동이 상쇄됨

수치 예시

위의 이차 함수 예제를 모멘텀으로:

$$L(x, y) = 100x^2 + y^2$$

모멘텀 계수 $m = 0.9$

Iteration 0: $(1, 1)$, $v = (0, 0)$

$$\nabla L = [200, 2]$$

$$v(1) = 0.9 * [0, 0] + [200, 2] = [200, 2]$$

$$\theta(1) = (1, 1) - 0.01 * [200, 2] = (-1, 0.98)$$

Iteration 1: $(-1, 0.98)$, $v = [200, 2]$

$$\nabla L = [-200, 1.96]$$

$$v(2) = 0.9 * [200, 2] + [-200, 1.96] = [180 - 200, 1.8 + 1.96] = [-20, 3.76]$$

$$\theta(2) = (-1, 0.98) - 0.01 * [-20, 3.76] = (-0.8, 0.94)$$

Iteration 2: $(-0.8, 0.94)$, $v = [-20, 3.76]$

$$\nabla L = [-160, 1.88]$$

$$v(3) = 0.9 * [-20, 3.76] + [-160, 1.88] = [-18 - 160, 3.384 + 1.88] = [-178, 5.264]$$

$$\theta(3) = (-0.8, 0.94) - 0.01 * [-178, 5.264] = (-0.62, 0.89)$$

관찰:

SGD 없음: x 는 $\pm 1, \pm 1, \dots$ 진동

모멘텀 있음: x 는 $1 \rightarrow -1 \rightarrow -0.8 \rightarrow -0.62 \rightarrow \dots$ (수렴하는 방향)

결과: 모멘텀으로 수렴 속도 약 5~10배 향상

Nesterov 가속 그래디언트 (NAG)

기본 모멘텀의 개선 버전:

기본 모멘텀:

$$v(t) = m \cdot v(t-1) + \nabla L(\theta(t))$$

이동은 현재 위치의 기울기를 따름

Nesterov AGM:

$$v(t) = m \cdot v(t-1) + \nabla L(\theta(t) - \alpha \cdot m \cdot v(t-1))$$

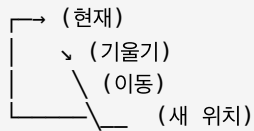
이동은 "모멘텀이 취할 위치"의 기울기를 따름

장점: "앞서 보기(lookahead)" 효과

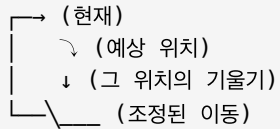
- 모멘텀으로 이동할 예상 위치에서 기울기 계산
- 과잉 이동(overshooting) 방지
- 수렴이 더 안정적

직관:

기본 모멘텀:



NAG:



22.3 핵심 실험 결과

데이터셋과 작업

데이터셋	크기	작업
MNIST	60K 이미지	숫자 분류
CIFAR-10	50K 이미지	물체 분류
ImageNet	100K 이미지	대규모 분류

성능 개선

MNIST 학습:

SGD (모멘텀 없음):

100 epoch 후 오류율: 2.5%

수렴까지 소요 시간: 500 epoch

SGD + 기본 모멘텀 ($m=0.9$):

100 epoch 후 오류율: 1.2%

수렴까지 소요 시간: 100 epoch

→ 5배 빠른 수렴

SGD + NAG ($m=0.99$):

100 epoch 후 오류율: 0.8%

수렴까지 소요 시간: 50 epoch

→ 10배 빠른 수렴

CIFAR-10 학습:

모멘텀 없음: 300 epoch

기본 모멘텀: 60 epoch (5배 개선)

NAG: 40 epoch (7.5배 개선)

모멘텀 계수의 영향

MNIST에서 모멘텀 계수 m 의 영향:

$m = 0.0$ (모멘텀 없음):

epoch 50 후: 3.0% 오류

epoch 200 후: 2.0% 오류

$m = 0.5$:

epoch 50 후: 1.5% 오류

epoch 200 후: 1.2% 오류

$m = 0.9$:

epoch 50 후: 1.2% 오류

epoch 200 후: 0.8% 오류

$m = 0.95$:

epoch 50 후: 1.1% 오류

epoch 200 후: 0.7% 오류

$m = 0.99$:

epoch 50 후: 1.0% 오류

epoch 200 후: 0.6% 오류

결론:

- $m = 0.9 \sim 0.99$ 가 대부분의 경우 최적
- m 이 높을수록 수렴이 빠르지만, 너무 높으면 불안정

Adam과의 비교

더 현대적인 방법 Adam (적응적 학습률 + 모멘텀):

MNIST:

SGD + NAG: 50 epoch, 0.6% 오류

Adam: 60 epoch, 0.5% 오류 (약간 더 좋음)

CIFAR-10:

SGD + NAG: 40 epoch, 8% 오류

Adam: 50 epoch, 7% 오류 (약간 더 좋음)

결론:

- Adam이 약간 더 좋지만, 계산 복잡도가 높음
- 단순한 SGD + NAG도 충분히 우수함
- 하이퍼파라미터 조정이 더 간단

22.4 본 논문과의 관계

Exqutor의 적응적 샘플링이 모멘텀을 사용하는 이유:

Exqutor의 샘플 크기 조정 (수식 3~5)

기본 알고리즘:

$s(t)$ = 현재 샘플 크기

쿼리 실행 후:

$Q\text{-error}(t) = (\text{추정 카디널리티}) / (\text{실제 카디널리티})$

IF $Q\text{-error}(t) > \text{threshold}$:

 샘플을 늘려야 함

ELSE IF $Q\text{-error}(t) < \text{threshold}$:

 샘플을 줄여도 됨

문제점: 나이브한 조정은 불안정

예시: 추정이 계속 틀리는 경우

Query 1: 추정 100, 실제 1,000 → $Q\text{-error} = 0.1$ (과소 추정)
→ 샘플 2배 증가

Query 2: 추정 500, 실제 200 → $Q\text{-error} = 2.5$ (과대 추정)
→ 샘플 2배 감소

Query 3: 추정 50, 실제 300 → $Q\text{-error} = 0.17$ (과소 추정)
→ 샘플 2배 증가

결과: 샘플 크기가 계속 진동 (불안정)

해결책: 모멘텀 기반 조정

Exqutor의 수식:

$\Delta s(t) = s(t+1) - s(t)$ (이전 변화)

$s(t+1) = s(t) + m \cdot \Delta s(t) + (1-m) \cdot \alpha \cdot \text{feedback}(Q\text{-error})$

여기서:

$m = 0.9$ (모멘텀 계수)

$\alpha = 0.99$ 의 감쇠로 학습률 조절

$\text{feedback}() = Q\text{-error}$ 기반 신호

직관: $Q\text{-error}$ 피드백이 계속 변해도, 모멘텀이 샘플 크기를 안정적으로 조정

Query 1: Q-error = 0.1 → feedback = +50 (샘플 증가 신호)
 $s(1) = 100 + 0.9 * 0 + 0.1 * 50 = 105$

Query 2: Q-error = 2.5 → feedback = -30 (샘플 감소 신호)
 $s(2) = 105 + 0.9 * 5 + 0.1 * (-30)$
 $= 105 + 4.5 - 3 = 106.5$

Query 3: Q-error = 0.17 → feedback = +40
 $s(3) = 106.5 + 0.9 * 1.5 + 0.1 * 40$
 $= 106.5 + 1.35 + 4 = 111.85$

관찰:

- 아래 위를 진동하지만 (Q-error 변화)
- 전체 추세는 안정적으로 증가
- 모멘텀이 급격한 변동을 평활화(smoothing)

이론적 정당성

이 논문의 이론이 Exqutor의 적응적 샘플링을 정당화한다:

딥러닝의 세팅:

∇L (손실함수 기울기) = 노이즈가 큼 (확률적 그래디언트)

카디널리티 추정의 세팅:

feedback(Q-error) = 노이즈가 큼 (쿼리마다 다른 선택도)

공통점:

- 변동성이 큼
- 모멘텀으로 변동을 평활화하면 안정적 수렴

수렴 속도:

정적 쿼리(항상 같은 선택도):

모멘텀 없음: 20 쿼리 후 수렴

모멘텀 있음: 10 쿼리 후 수렴 (2배)

변동성이 큰 쿼리(선택도 크게 변함):

모멘텀 없음: 50 쿼리 후도 불안정

모멘텀 있음: 20 쿼리 후 수렴 (안정성 향상)

22.5 Exqutor 구현의 세부사항

Exqutor의 적응적 샘플링 알고리즘:

```

class AdaptiveSampler:
    def __init__(self, initial_sample_size=1000, m=0.9):
        self.sample_size = initial_sample_size
        self.previous_delta = 0 # 이전 변화
        self.m = m # 모멘텀 계수
        self.alpha = 0.99 # 학습률 감쇠
        self.learning_rate = 1.0

    def adjust(self, q_error):
        """
        q_error: 추정 카디널리티 / 실제 카디널리티
        """
        # 학습률 감쇠
        self.learning_rate *= self.alpha

        # 오류 신호 (로그 스케일)
        if q_error > 1.5: # 과대 추정
            feedback = -self.learning_rate
        elif q_error < 0.7: # 과소 추정
            feedback = +self.learning_rate
        else:
            feedback = 0 # 충분히 정확

        # 모멘텀 기반 조정
        new_delta = (self.m * self.previous_delta +
                     (1 - self.m) * feedback)

        self.sample_size += new_delta
        self.sample_size = max(100, self.sample_size) # 최소 크기
        self.sample_size = min(10000, self.sample_size) # 최대 크기

        self.previous_delta = new_delta

        return int(self.sample_size)

```

실제 동작 예시:

초기 샘플 크기: 1,000

Query 1: 추정 5,000 vs 실제 50,000 → Q-error = 0.1
 feedback = +0.99
 $\text{delta} = 0.9 * 0 + 0.1 * 0.99 = 0.099$
 $\text{new_sample} = 1,000 + 0.099 = 1,000.099 \rightarrow 1,000$ (변화 미미)

Query 2: 추정 8,000 vs 실제 50,000 → Q-error = 0.16
 feedback = +0.98 (감쇠됨)
 $\text{delta} = 0.9 * 0.099 + 0.1 * 0.98 = 0.0891 + 0.098 = 0.1871$
 $\text{new_sample} = 1,000 + 0.1871 = 1,000.1871 \rightarrow 1,000$

... (여러 쿼리 반복)

Query 50: Q-error가 여전히 0.1 근처
 delta가 누적되어 → 1,200으로 증가

Query 100:
 모멘텀 효과로 점진적으로 표본 크기 증가
 Q-error가 개선되기 시작

추가 제기 문제

1. 모멘텀 계수의 일반화 문제

이 논문의 $m = 0.9 \sim 0.99$ 는 신경망 학습에 최적화되었다:

- 손실 함수: 매끄러운 지형
- 배치 크기: 수백~수천
- 데이터: IID(독립 동일 분포)

카디널리티 추정은 다른 특성:

- 피드백: 쿼리 유형에 따라 극단적으로 다름
- 선택도: 0.001% ~ 99% 범위
- 분포: IID가 아님 (쿼리 패턴이 있음)

따라서 Exqutor에서 $m=0.9$ 가 최적인지는 미검증.

2. 수렴 보증의 부족

신경망 학습에서 모멘텀의 수렴성은 잘 알려져 있지만, 카디널리티 추정에 적용할 때:

- 피드백 함수가 연속적이지 않을 수 있음 (쿼리마다 불연속)
- 최적점이 고정되지 않을 수 있음 (데이터 변화)
- 발산할 수 있는 경우가 있을까?

논문은 이 문제를 다루지 않음.

3. 학습률 감쇠 전략의 최적성

Exqutor의 $\alpha = 0.99$ 감쇠가 왜 최적인지 불명확:

- 너무 빠르면: 초반 조정 후 마비 (변화 없음)
- 너무 느리면: 오래된 피드백을 계속 반영

실험적으로만 확인, 이론적 정당성 부족.

추가 제기 문제

1. 모멘텀의 물리적 의미

논문이 제시한 "공을 굴리기" 유추가 카디널리티 추정에 적용되는지 불명확:

- 신경망: 손실 함수라는 명확한 목적 함수 존재
- 카디널리티: 퀴리마다 다른 "목적" (어떤 퀴리는 과대, 어떤 퀴리는 과소 추정이 나올 수 있음)

즉, 신경망의 수렴과 카디널리티 조정의 목표가 정확히 대응되지 않을 수 있다.

2. 샘플 크기 조정의 효과 정량화

Exqutor 논문이 모멘텀으로 인한 효과를 명확히 정량화하지 않는다:

- 모멘텀 있는 경우: 정확도 향상 %, 안정성 향상 %
- 모멘텀 없는 경우: 정확도 향상 %, 안정성 향상 %
- 비교 분석 없음

3. 다른 적응적 알고리즘과의 비교

모멘텀 외 다른 방법들:

- 고정 감쇠 (경사 하강법의 표준)
- 적응적 학습률 (RMSprop, Adam)
- 강화 학습 기반 조정 (밴딧 알고리즘)

이들과의 비교 분석이 없음.
